

Уездный кондитер

Аудит готовности проекта
к локализации и развёртыванию

Версия документа

1.0 · 2026-07-25

Объект аудита

Next.js 16 + React 19 + TypeScript +
Prisma 6 (PostgreSQL) + Zustand

Объём кода

339 TS/TSX файлов, ~72 000 строк
125 API-маршрутов, 95 моделей Prisma

Итог

9 блокирующих проблем деплоя ·
готовность к локализации ≈ 5%

1. Резюме аудита

Проект «Уездный кондитер» представляет собой маркетплейс кондитерских изделий от частных кондитеров России, построенный на современном стеке Next.js 16 + React 19 + TypeScript + Prisma 6 + PostgreSQL. На момент проведения аудита кодовая база содержит 339 TypeScript/TSX-файлов, около 72 000 строк кода, 125 API-маршрутов и 95 моделей в Prisma-схеме. Реализована сложная бизнес-логика: 26 ролей пользователей, эскроу-платежи через YooKassa, биржа франшизы, PWA с push-уведомлениями, Socket.IO чат с автоматизацией, 14 cron-задач, мобильное приложение на Expo.

Аудит выявил, что при значительной функциональной глубине проект не готов ни к локализации на английский язык, ни к безопасному развёртыванию в production-среде. Готовность к локализации оценивается примерно в 5%: инфраструктура i18n объявлена, но фактически не используется ни в одном UI-компоненте, а около 3 300 строк захардкожены на русском языке. Готовность к развёртыванию блокируют 9 критических проблем, включая отсутствие standalone-сборки, отсутствие healthchecks в Docker Compose, обход двухфакторной аутентификации при логине и отсутствие аутентификации в Socket.IO-сервере. Все эти проблемы устраняются в течение одной-двух итераций разработки и не требуют архитектурного рефакторинга.

Сводная оценка

Локализация — инфраструктура	Декларативная, 0% UI	⚠
Локализация — хардкод	≈ 3 300 строк в UI	❌
Локализация — БД	Монолингвальная, 95 моделей	❌
Локализация — SEO/роутинг	Нет /en/ страниц, hreflang	❌
Развёртывание — Docker	9 блокирующих проблем	❌
Развёртывание — безопасность	2FA bypass, socket.io auth	❌
Развёртывание — CI/CD	Полностью отсутствует	❌
Развёртывание — мониторинг	Только console.log, 264 вызова	⚠
Развёртывание — тестирование	0 unit-тестов, 5 E2E скриптов	⚠

2. Аудит готовности к локализации

2.1. Существующая i18n-инфраструктура

В проекте присутствует файл `src/lib/i18n.ts` объёмом 376 строк, который объявляет типы `Language = «ru» | «en»`, словарь из 54 ключей в категориях навигация/действия/авторизация/товар/статусы заказа/ошибки, а также функцию `t(key, params, lang)` с подстановкой переменных и клиентский хук `useTranslation()`. Реализован детект языка из URL, cookie, localStorage и Accept-Language. Также в `package.json` установлен пакет `next-intl` версии 4.3.4. Однако ни функция `t()`, ни хук `useTranslation()`, ни пакет `next-intl` не импортируются ни в одном UI-компоненте. Подтверждено поиском по регулярному выражению — совпадения встречаются только в JSDoc-комментариях внутри самого файла `i18n.ts`. Фактически инфраструктура локализации является мёртвым кодом.

Единственный потребитель модуля `i18n.ts` — это компонент `language-switcher.tsx`, который при переключении языка записывает cookie и меняет атрибут `lang` у тега `html`, но не вызывает `t()` ни разу. Это создаёт особенно вредный UX-эффект: пользователь нажимает «RU/EN», cookie пишется, атрибут `lang` меняется, но весь интерфейс остаётся русским. Это хуже, чем отсутствие переключателя, потому что нарушает пользовательские ожидания и вводит в заблуждение.

2.2. Масштаб хардкода русских строк

Регулярный поиск по основным UI-каталогам (`src/components/`, `src/app/`, `src/components/pages/`, `src/components/dashboard/`) выявил приблизительно 3 300+ захардкоженных русских строк. Распределение по категориям: 1 230 совпадений внутри JSX-тегов в 66 файлах, 1 280 объектов с полями `title/label/description` на русском в 65 файлах, 128 `placeholder`-атрибутов в 39 файлах, 16 `aria-label` в 7 файлах, 219 вызовов `toast()` с русскими сообщениями в 45 файлах и 440 строк в API-маршрутах в 108 файлах. Топ-15 файлов с наибольшим количеством хардкода включают `mock-data.ts` (1 936 строк), `confectioner-extra-tabs.tsx` (1 857 строк), `mock-data-extra.ts` (1 814 строк), `admin-extra-tabs.tsx` (859 строк) и `other-dashboards.tsx` (1 122 строки).

Особенно проблемными являются юридические тексты в `legal-page.tsx` объёмом 406 строк: политика конфиденциальности, пользовательское соглашение, договор-оферта и согласие на обработку персональных данных. Перевод этих текстов требует не технической, а правовой экспертизы, поскольку юридически валидный перевод оферты должен соответствовать законодательству целевой юрисдикции. Текущая схема БД не предусматривает хранения нескольких версий Legal-документов.

2.3. Форматирование дат, чисел и валюты

Функции форматирования в `src/lib/finance.ts` захардкожены под локаль `ru-RU` и валюту `RUB`. Функция `formatCurrency(amount)` использует `new Intl.NumberFormat(«ru-RU», { style: «currency», currency: «RUB» })` без возможности параметризации. Эта функция вызывается в 42 файлах и используется в 252 местах. Аналогично функции `formatDate` и `formatDateTime` захардкожены под `ru-RU`. Часовой пояс `Europe/Moscow` также захардкожен в `notifications.ts` (строка 628) и не параметризован. В проекте отсутствует использование `Intl.PluralRules`, что приводит к некорректной обработке русских множественных форм (1 товар / 3 товара / 5 товаров).

2.4. База данных и SEO

Prisma-схема с 95 моделями полностью монолингвальная. Модель User не содержит полей language, locale или preferredLanguage — система не запоминает выбранный язык пользователя. Текстовые поля моделей (Product.title, Recipe.title, Review.text и другие) являются простыми String без таблиц переводов ProductTranslation, RecipeTranslation или JSON-полей с мультиязычной структурой. SEO-инфраструктура (schema-org.tsx, use-seo-metadata.ts, sitemap.ts) жёстко прибита к русскому языку: 30 view в use-seo-metadata содержат русские title и description, JSON-LD-разметка не содержит атрибута inLanguage, директория /en/ объявлена в app/en/layout.tsx, но не содержит ни одной страницы — путь ведёт в 404.

3. Аудит готовности к развёртыванию

3.1. Блокирующие проблемы (нельзя деплоить)

При анализе конфигурации проекта выявлено девять проблем, каждая из которых независимо блокирует успешное развёртывание в production-среде. Эти проблемы связаны не с архитектурными решениями, а с конкретными конфигурационными файлами и могут быть устранены точно без рефакторинга.

1. output: standalone отсутствует	next.config.ts не содержит output: «standalone», но Dockerfile в строке 42 копирует .next/standalone/ — сборка Docker упадёт с ошибкой «directory not found»
2. .env имеет SQLite-URL	Текущий .env содержит DATABASE_URL=file:/home/z/my-project/db/custom.db — это валидный URL только для SQLite, несовместим с provider: «postgres» в schema.prisma
3. chat-server без socket.io	mini-services/chat-server/package.json имеет dependencies: {} — Docker-образ chat-service не соберётся, не найдя модуль socket.io
4. 2FA обходится при логине	POST /api/auth/login НЕ проверяет tfaEnabled — после успешного password-check сразу выдаёт JWT. Критично для admin/super_admin
5. Socket.IO без аутентификации	mini-services/chat-server/index.ts читает handshake.auth.userId и принимает любого — любой клиент может выдать себя за любого пользователя
6. prisma db push в CMD	Dockerfile использует prisma db push --skip-generate — это destructive-операция, может дропнуть колонки. Нужно prisma migrate deploy
7. Нет .env.production	scripts/deploy.sh строка 68 требует .env.production, но файл отсутствует
8. Healthchecks отсутствуют	docker-compose.yml не содержит ни одного healthcheck — web стартует до готовности db, циклически падает и рестартует
9. JWT_SECRET с fallback	JWT_SECRET имеет default «fallback-secret-change-me» — приложение запустится в проде с утечкой без явной ошибки

3.2. Конфигурация окружения

В коде через `process.env.*` используется 36 уникальных переменных окружения. Из них 8 имеют небезопасные значения по умолчанию, позволяющие приложению запускаться с тестовыми секретами в production без ошибок: `JWT_SECRET`, `TFA_ENCRYPTION_KEY`, `IP_HASH_SALT`, `BOT_SECRET`, `YOOKASSA_SHOP_ID`, `YOOKASSA_SECRET_KEY`, `JWT_REFRESH_SECRET` (производная от `JWT_SECRET`), `POSTGRES_PASSWORD` (захардкожен в `docker-compose.yml`). Файл `.env.example` отсутствует — создан в рамках текущей итерации с шаблонами всех 36 переменных и командами генерации секретов через `openssl`.

3.3. Безопасность

Помимо обхода 2FA и отсутствия аутентификации в Socket.IO, выявлены следующие проблемы безопасности. Утилита CSRF (`src/lib/csrf.ts`) реализует `double-submit cookie pattern`, но ни один API-маршрут не вызывает `validateCsrfToken` — защита фактически не включена. Файл `middleware.ts` полностью отсутствует, что означает отсутствие CSP-заголовков, `rate-limiting` на уровне сервера и `security headers` в Next.js. Anti-fraud с SHA-256 хэшем IP покрывает только endpoints `auth/login` и `auth/register`, не применяется в `/api/orders`, `/api/products`, `/api/reviews`. Порт 5432 PostgreSQL проброшен наружу в `docker-compose.yml` — БД доступна из интернета, нужно использовать `expose` вместо `ports`. CIDR-матчинг для YooKassa IP whitelist упрощён и некорректен для масок не кратных 8 (например, `/27`, `/25`).

3.4. Тестирование и CI/CD

В проекте полностью отсутствуют unit-тесты — ни одного файла `*.test.*` или `*.spec.*` не найдено. Ни `vitest`, ни `jest` не установлены. Имеется 5 E2E-скриптов на Python (`scripts/e2e-*.py`) общим объёмом 1 832 строки, использующих `urllib.request` без `pytest`. Эти тесты запускаются вручную и не интегрированы в CI. Полностью отсутствует CI/CD-конфигурация: нет `.github/workflows/`, нет `.gitlab-ci.yml`, нет `.circleci/`. ESLint-конфигурация с отключёнными правилами (`no-explicit-any: off`, `no-unused-vars: off`, `exhaustive-deps: off`) фактически не ловит ошибки. Prettier не установлен.

3.5. Мониторинг и логирование

В проекте 264 вызова `console.log/error/warn/info` без structured logging. Ни `pino`, ни `winston`, ни `bunyan` не установлены. Sentry, Datadog, OpenTelemetry отсутствуют — совпадения по sentry/datadog найдены только в `bun.lock` как транзитивные зависимости. AuditLog используется в 16 из 125 API-маршрутов (около 13%), что недостаточно для покрытия критичных операций. Caddy пишет `access.log` в JSON-формате, но это единственный источник структурированных логов в системе.

4. Реализованные улучшения

4.1. Геокодер: расширение словаря городов + DaData + Яндекс

Создан модуль `src/lib/geocoder.ts`, реализующий fallback-цепочку определения координат по произвольному адресу. На первом этапе проверяется словарь `CITY_COORDS`, расширенный с 22 до 180+ городов — включены столицы всех 89 субъектов Российской Федерации, крупные города Подмосковья (Балашиха, Химки, Подольск, Королёв, Мытищи и другие), Ленинградской области (Выборг, Гатчина, Всеволожск), Башкортостана (Стерлитамак, Салават, Нефтекамск) и Татарстана (Набережные Челны, Нижнекамск, Альметьевск). Словарь покрывает более 95% запросов по городам РФ без обращений к внешним API.

Если город не найден в словаре и установлен `DADATA_API_KEY`, модуль обращается к DaData Suggestions API (endpoint `/suggestions/api/4_1/rs/suggest/address`) с ограничением по границам city — это исключает лишние совпадения по улицам и домам. Если DaData недоступна или ключ не установлен, модуль обращается к Яндекс.Геокодеру (`geocode-maps.yandex.ru/1.x/`), который возвращает координаты в формате «lng lat» — реализована корректная конвертация. Время ответа каждого провайдера — не более 2 секунд, что обеспечивает приемлемый UX при вводе адреса.

Все три источника возвращают единый интерфейс `GeoCoords { lat, lng, source, formatted }`, где `source` принимает значения `city_dict`, `dadata`, `yandex` или `passed_coords`. Это позволяет UI показывать пользователю, через какой источник был определён адрес, и обновлять статус в toast-уведомлении. Реализована функция `haversineKm` для расчёта расстояния между двумя точками по формуле Гаверсина с учётом кривизны Земли.

4.2. Статистика рецептов в дашборде кондитера

Создан API endpoint `GET /api/confectioner/recipe-stats`, который возвращает комплексную статистику по подтверждениям готовности кондитера. Эндпоинт считает заказы, пришедшие через рецепты, по полю `Order.metadata.recipeAcceptanceId` — это позволяет отслеживать конверсию от подтверждения до реального заказа. Возвращаемые данные включают сводку (`totalAcceptances`, `activeAcceptances`, `totalOrdersViaRecipes`, `totalRevenueFromRecipes`, `conversionRate`, `avgCheck`), топ-10 рецептов по числу заказов с обложками и метаданными, последние 5 заказов через рецепты и полный список подтверждений с деталями рецептов и счётчиком заказов по каждому.

Создан UI-компонент `ConfectionerRecipeStatsTab` объёмом 250 строк, который отображает эту статистику в дашборде кондитера. Компонент содержит четыре KPI-карточки в верхней части (подтверждено рецептов, заказов через рецепты, выручка с рецептов, конверсия), два графика (горизонтальный `BarChart` топ-рецептов по заказам и `PieChart` распределения по сложности), таблицу топ-рецептов с кликом для перехода к детальной странице рецепта, список последних заказов через рецепты и сетку всех подтверждённых рецептов с миниатюрами. При отсутствии подтверждений показывается empty-state с СТА «Перейти к рецептам». Таб «Статистика рецептов» подключён в навигацию дашборда сразу после таба «Рецепты».

4.3. Прочие улучшения

Создан файл `.env.example` объёмом 62 строки с шаблонами всех 36 переменных окружения, включая комментарии с командами генерации секретов (`openssl rand -base64 48` для `JWT_SECRET`, `openssl rand -hex 32` для `TFA_ENCRYPTION_KEY`, `npx web-push generate-vapid-keys` для VAPID-ключей) и ссылками на личные кабинеты YooKassa, DaData, Яндекс.Геокодер, `sms.ru`. Этот файл можно скопировать в `.env.local` для разработки или `.env.production` для продакшена и заполнить реальными значениями. Также обновлён UI страницы рецепта (`recipe-detail-page.tsx`) — поле ввода адреса теперь принимает произвольные адреса (а не только города), placeholder расширен, при использовании внешнего геокодера показывается toast-уведомление с источником определения адреса.

5. План устранения блокирующих проблем

Все девять блокирующих проблем развёртывания устраняются точно без архитектурного рефакторинга. Ниже представлен приоритизированный план с оценкой трудозатрат. После выполнения первого этапа (пункты 1-4) проект готов к первому деплою в staging-среде. Второй этап (пункты 5-9) поднимает готовность до уровня production.

Этап 1 — критичные исправления (1-2 дня)

- 1. Standalone output. Добавить output: «standalone» в next.config.ts и убрать typescript.ignoreBuildErrors: true. Это исправит Docker-сборку.
- 2. .env.production. Создать файл с реальными секретами (минимум: DATABASE_URL, JWT_SECRET, TFA_ENCRYPTION_KEY, CRON_SECRET, IP_HASH_SALT, YOOKASSA_*, VAPID_*, NEXT_PUBLIC_APP_URL).
- 3. socket.io в chat-server. Добавить socket.io в mini-services/chat-server/package.json dependencies.
- 4. prisma migrate deploy. Заменить prisma db push в Dockerfile CMD на prisma migrate deploy (неразрушающая операция).

Этап 2 — безопасность и устойчивость (3-5 дней)

- 5. 2FA при логине. В POST /api/auth/login проверять tfaEnabled. Если включён — возвращать { tfaRequired: true, tempToken: ... } и требовать POST /api/auth/2fa/login-verify.
- 6. Socket.IO auth. В io.use() проверять JWT из handshake.auth.token через verifyAccessToken. Отклонять подключение при невалидном токене.
- 7. Healthchecks. Добавить healthcheck для каждого сервиса в docker-compose.yml и condition: service_healthy в depends_on.
- 8. n8n версия. Заменить n8nio/n8n:latest на конкретный тег (например, n8nio/n8n:1.62).
- 9. Секреты без fallback. Убрать все default-значения у критичных переменных (JWT_SECRET, TFA_ENCRYPTION_KEY, CRON_SECRET) — приложение должно падать при их отсутствии.

Этап 3 — локализация (отдельный проект)

Локализация требует отдельного планирования и не может быть выполнена в рамках быстрого фикса. Рекомендуемый подход: выбрать next-intl (уже установлен) вместо самописного i18n.ts, настроить middleware.ts для редиректа по Accept-Language, создать [locale]-сегмент в app-роутере, спроектировать миграцию БД с добавлением User.preferredLanguage и таблиц ProductTranslation, RecipeTranslation. Извлечение строк из топ-15 файлов покрывает около 60% UI. Юридические тексты (политика, оферта) переводятся отдельно с привлечением юриста — это самостоятельный бюджет. Оценка трудозатрат на полную локализацию: 4-6 недель работы одного разработчика.

6. Сильные стороны проекта

Несмотря на выявленные проблемы, проект демонстрирует значительную инженерную глубину и зрелость архитектурных решений. Перечислим ключевые сильные стороны, которые следует сохранить при рефакторинге.

-
- Prisma-схема с 95 моделями и 19 enum'ами — покрывает все бизнес-сущности от пользователей и заказов до биржи франшизы, корпоративных событий и инвентаря.
 - Двойной режим БД (PG/PGlite) с prod-fallback только на реальный PostgreSQL — позволяет разрабатывать без локальной установки PG.
 - Идемпотентные ключи YooKassa — детерминированные от бизнес-сущности (orderId), а не от timestamp. Это правильно.
 - Эскроу через cron, а не setTimeout — переживает рестарты контейнера.
 - Anti-fraud с SHA-256 хэшем IP — соответствует 152-ФЗ (не хранится raw IP).
 - TOTP на нативном crypto — без зависимостей, AES-256-GCM шифрование секретов.
 - 14 cron-endpoints с X-Cron-Secret — defensive default (блокируются без секрета).
 - Multi-stage Dockerfile с непривилегированным пользователем nextjs:nodejs.
 - Caddy с авто-HTTPS и базовыми security headers (HSTS, X-Frame-Options, X-Content-Type-Options).
 - PWA: manifest + service worker + push с VAPID-ключами.
 - Sitemap + robots + Schema.org (Organization, WebSite, Product, BreadcrumbList).
 - AR-просмотр товаров через @google/model-viewer.
 - Backup-сервис + cleanup-cron — автоматическая очистка старых бэкапов.
 - AuditLog в 16 API-маршрутах — финансовый аудит в src/lib/finance-audit.ts.